Technical audit — Site Security module

Summary of the audit of the local code

Title	Technical audit — Site Security module
Status	Summary of the audit of the local code
Author	Tania ZABRE
Date	28 October 2026
Scope	• Backend of the Site Security module (Spring Boot / Java); • Frontend of the Site Security module (React); • Visible interactions with Visitor Management; • Entities, controllers, services, repositories, DTOs, hooks and components involved.
Limits	• Static analysis of the local code only. • No real deployment is assumed. • No live HTTP test is reported as done. • External systems that may exist outside the repository are not covered.

How to read this document

- **Fact checked in the code:** seen directly in the code, with file and line numbers.
- **Deduced risk:** a possible consequence of those facts. It was not reproduced in a live test.
- **To confirm:** needs a check in the real environment, or a business decision.

This is an audit summary. It is not a fix plan, not an ADR, not an architecture proposal and not a report of tests that were run. No fix is described as done.

Short names used for file paths

Short name	Full path
BR	`backend\src\main\java\com\infineonbit\sustainablefarm\`
BE	`BR\modules\sitesecurity\ (backend, Site Security module)`
BT	`backend\src\test\java\com\infineonbit\sustainablefarm\modules\sitesecurity\`
FE	`frontend\src\features\sitesecurity\ (frontend, Site Security module)`

2. Executive conclusion

> The Site Security module is currently a working prototype to view and create access records. It is not yet an access-control system that is ready for production.

Main findings

- No authentication or authorization mechanism was found in the code that was inspected.

- The Site Security endpoints expose sensitive read operations and one operation that creates a credential.
- The “credentials” are descriptive records. They are not real badges, PINs, QR codes or tokens.
- The logs are filled with demo data. No real reader, gate or physical device feed is visible in the repository.
- Credential and log data can contain personal and operational information that is sensitive.
- No direct link between Site Security and Visitor Management is visible.
- There is no pagination for zones, credentials, gates or logs.
- Several numbers and screen elements are fixed (hard-coded) or simulated.
- In development, the frontend can replace an API error with fixture (fake) data.
- The existing tests mostly cover the service. They do not cover HTTP access control, roles, concurrency or data leaks.

References

- `backend\...\modules\sitesecurity\controller\SiteSecurityController.java:12-43`
- `backend\...\modules\sitesecurity\service\SiteSecurityService.java:27-243`
- `frontend\src\features\sitesecurity\api\sitesecurityApi.js:1-61`
- `frontend\src\features\sitesecurity\components\SiteSecurityApp.jsx:1-50`

3. What the module does today

This section describes only what exists in the inspected code.

3.1 Dashboard

The dashboard shows statistics, a drawing of the perimeter, access points, a feed of events, the perimeter status and shortcuts to the other screens.

Fact checked in the code. Several numbers are fixed in the frontend. The same numbers are also fixed in the backend:

Indicator	Hard-coded value
Active Access Points	4
Controlled Zones	3
Active Visitors	2
Alerts	1

> These numbers cannot be treated as a reliable picture of the real state of the database without further checks.

References

- `FE\screens\Dashboard.jsx:1-192 (screen); statistics at lines 15-24`
- `BE\service\SiteSecurityService.java:79-87 (same statistics in the backend)`

3.2 Zones / Perimeter

The screen lets the user select a zone. It shows the zone status, the number of authorized users, the number of entry points and the requirements. It also opens the log activity for that zone. **Fact checked in the code:** the map is mostly defined in the frontend.

> The frontend map can be different from the data really stored in the database.

References

- `FE\screens\ZonesScreen.jsx:1-152`
- `FE\components\PerimeterMap.jsx:8-32 and 128-150`

3.3 Access Control

For each entry the screen shows name, type, level, validity, last activity and status. A form (AddAccessModal) is used to create a new entry.

Fact checked in the code. The frontend creates a local identifier with `Date.now()`. The backend creates its own identifier.

References

- `FE\screens\AccessControl.jsx:1-108`
- `FE\components\AddAccessModal.jsx:1-176 (local identifier: lines 36-48)`
- `BE\service\SiteSecurityService.java:133-140 (identifier created in the backend)`

3.4 Access Logs

The screen has the filters all, approved, denied, visitors, staff and service. It shows the number of results, a table and a detail drawer.

Fact checked in the code. The “Mark as reviewed” button does not seem to save any business action.

> The inspected code shows that this action only closes the interface. No API call and no endpoint that saves it was found.

References

- `FE\screens\AccessLogScreen.jsx:1-113`
- `FE\components\LogDrawer.jsx:120-130`

4. Existing endpoints

Fact checked in the code. These endpoints were found:

Method	Path	What it does
GET	/api/v1/sitesecurity/overview	Reads the overview
GET	/api/v1/sitesecurity/zones	Reads the zones
GET	/api/v1/sitesecurity/credentials	Reads the credentials
POST	/api/v1/sitesecurity/credentials	Creates a credential
GET	/api/v1/sitesecurity/logs	Reads the logs

References

- `BE\controller\SiteSecurityController.java:18-43`

No visible endpoint exists to:

- edit, delete, suspend or revoke a credential;
- create, edit or delete a log;
- mark a log as handled.

5. Data model

5.1 Gates

Table `site_security_gates`. Fields: id, name, shortName, zone, status, detail. **Fact checked in the code:** zone is stored as a text string. No JPA relation to ZoneEntity was found.

References

- `BE\entity\GateEntity.java:13-24`

5.2 Zones

Table `site_security_zones`. Fields: id, name, tagline, authorizedUsers, entryPoints, status, description, requirements. **Fact checked in the code:** requirements uses an ElementCollection (line 26). No direct reverse relation to gates, credentials or logs was found.

References

- `BE\entity\ZoneEntity.java:15-30 (ElementCollection: line 26)`

5.3 Credentials

Table `site_security_users`. **Fact checked in the code:** the model only holds descriptive data: name, role, type, level, validity, status, initials and last activity.

It does **not** hold: badge, PIN, QR code, token, hash, creation date, revocation, approver, visitorId, registrationId.

> Today, the word “credential” means a descriptive authorization record. It is not a technical way to enter a place.

References

- `BE\entity\CredentialUserEntity.java:13-25`

5.4 Logs

Table `site_security_logs`. Fields: user, reference, time, date, zone, gate, method, action, status, note. **Facts checked in the code:**

- date and time are text strings;
- no typed server timestamp was found;
- no authenticated source is stored;
- no hash, signature or append-only mechanism was found.

References

- `BE\entity\LogEntryEntity.java:13-31`

6. API security

Facts checked in the code

- no Spring Security dependency was found in `backend\pom.xml`;
- no authentication filter was found;
- no role or authorization annotation was found in SiteSecurityController.java;
- the Axios client does not send an Authorization header.

References

- `backend\pom.xml:37-40`
- `BE\controller\SiteSecurityController.java:12-43`
- `frontend\src\shared\api\client.js:1-34`

> The analysis of the repository shows no authentication or authorization mechanism for these endpoints. Whether anonymous calls really work must be confirmed by an HTTP test on a running instance.

Deduced risks

- unauthorized reading of zones;
- unauthorized reading of credentials;
- unauthorized reading of logs;
- unauthorized creation of a credential.

No live test was run. These risks must not be read as attacks that were seen.

7. Creating credentials

Existing checks (facts checked in the code)

- name is required, maximum length 100;
- role is required, maximum length 100;
- type, level and validUntil are required;
- type and status must be in an allowed list;
- level must match an existing zone;
- validUntil must be an ISO date or “Permanent”;
- the date must not be in the past.

Missing checks (not found in the inspected code)

- who the creator is, and the creator’s role;
- right to give the level “critical”;
- right to create “Permanent” access;
- link with a Visitor;
- consistency between type and level;
- approval, revocation and automatic expiry.

> No real access secret is stored today. The problem is that a real credential model does not exist. It is not that an existing token is stored in clear text.

References

- `BE\controller\SiteSecurityController.java:33-36`
- `BE\service\SiteSecurityService.java:107-150`
- `BE\dto\SiteSecurityDtos.java:72-84`

To confirm: the exact business rules for “critical”, “Permanent” and visitors.

8. Access logs

Facts checked in the code

- logs are created at startup if the table is empty;
- the values are demo data;
- there is no POST /logs endpoint;
- no real reader, gate or physical device source is visible in the repository;
- no typed UTC server timestamp is visible;
- no append-only or integrity mechanism is visible.

> No real flow that creates an access event was found in the inspected code. An external integration outside this repository cannot be ruled out, however.

References

- `BE\service\SiteSecurityService.java:27-76`
- `BE\controller\SiteSecurityController.java:18-43`

9. Fixtures and fallbacks

Backend (facts checked in the code)

- SiteSecurityService uses `@PostConstruct` to load initial data;
- this loading is not protected by `@Profile("dev")` in the inspected code.

References

- `BE\service\SiteSecurityService.java:27-76`
- `For comparison: BR\core\config\DevDataSeeder.java:32-41`

Frontend (facts checked in the code)

- in DEV mode, GET errors can return fixture data;
- the code prints a console.warn;
- no clear message for the user was found;
- the fallback only runs when `import.meta.env.DEV` is true. It must not be presented as a normal production fallback.

References

- `FE\api\sitesecurityApi.js:1-53`

Deduced risks

- an API failure can be hidden;
- fake data can be taken for real data;
- a false feeling of security.

To confirm: which profiles are really used at runtime (deployment environment).

10. Personal data and GDPR

Data exposed by the DTOs (facts checked in the code)

Object	Fields exposed
Credentials	name, role, type, level, validity, status, last activity
Logs	user, date/time, zone, gate, method, action, status, note

References

- `BE\dto\SiteSecurityDtos.java:58-72 (credentials)`
- `BE\dto\SiteSecurityDtos.java:86-105 (logs)`

The inspected code does not show:

- filtering by role; data minimization; retention rules;
- anonymization; deletion;
- an audit of who views the data;
- restriction by site or organization.

> These points are a possible data-protection issue. They must be reviewed with the security, legal and business owners.

11. Concurrency, integrity and pagination

Concurrency

Fact checked in the code: creation first calls `existsByName` and then `save`. No unique constraint is clearly visible. **Deduced risk:** two creations at the same time could both succeed.

> Risk found in the code. A concurrent test was not run in this audit.

References

- `BE\service\SiteSecurityService.java:109-140`
- `BE\entity\CredentialUserEntity.java:13-25`

Pagination

Fact checked in the code: the services use `findAll`. No pagination is visible for zones, credentials, gates or logs.

Deduced consequences

- responses can become very large;
- no filter by time;
- no limit on size;
- no guaranteed explicit sorting.

References

- `BE\service\SiteSecurityService.java:91-94`

12. HTTP errors

Facts checked in the code: IllegalArgumentException is turned into 409 Conflict. Normally, an invalid input should be 400 Bad Request, a missing resource 404 Not Found, and only a real conflict 409 Conflict.

> The error mapping does not separate the error types well enough. The exact HTTP behavior must be confirmed with an integration test.

References

- `BE\exception\SiteSecurityExceptionHandler.java:12-33`

13. Existing tests and limits of the audit

Existing test file: `BT\service\SiteSecurityServiceTest.java`. It covers:

- initialization; valid creation; name that already exists;
- unknown zone; invalid type; invalid status;
- past date; invalid date; Permanent.

Scenarios not covered

- authentication; authorization; 401; 403; MockMvc endpoint tests;
- credential leaks; log leaks;
- concurrency; unique constraint; pagination;
- real log creation; revocation; expiry;
- Visitor Management integration; frontend fallback; accessibility.

> This audit is a static analysis. No test result, passed or failed, is reported in this document.

14. Strengths seen

- clear controller / service / repository structure;
- DTOs separate from entities;
- basic validation of the fields;
- allowed lists for some types and statuses;
- Spring Data repositories;
- no SQL string concatenation found;
- React escapes displayed values by default;
- loading and error states exist;
- unit tests for the service already exist;
- frontend fixtures are clearly limited to DEV.

15. Main weaknesses

Weaknesses are ranked by indicative priority. The “Type” column shows whether the point is checked in the code, deduced, or to be confirmed.

Level	No.	Weakness	Type	Section
CRITICAL	1	No authentication or authorization visible	Fact checked; real effect to confirm by HTTP test	6
CRITICAL	2	Access can be created without any authority check	Fact checked (checks missing from the code)	7
CRITICAL	3	Possible exposure of security and personal data	Deduced risk; review with security, legal and business	10
HIGH	4	Demo logs and no creation of real events	Fact checked; external integration to confirm	8
HIGH	5	No link with Visitor Management	Fact checked (link not found in the repository)	5
HIGH	6	No full credential life cycle	Fact checked	4, 5, 7
HIGH	7	Backend seed not limited to the dev profile	Fact checked in inspected code; deployment to confirm	9
MEDIUM	8	No pagination	Fact checked	11
MEDIUM	9	Fixed statistics	Fact checked	3.1
MEDIUM	10	Concurrency risk; no visible unique constraint	Deduced risk; not reproduced	11
MEDIUM	11	HTTP error codes not distinct enough	Fact checked; HTTP behavior to confirm	12
MEDIUM	12	Accessibility incomplete	To confirm; not covered by tests	13
MEDIUM	13	“Mark as reviewed” does not save	Fact checked	3.4
MEDIUM	14	Frontend fallback can hide an outage in development	Fact checked; deduced risk	9

16. Top 5 priorities

These are recommendations from the audit. They are not a detailed fix plan, and none of them is done today.

1. Add authentication and authorization in the backend.

2. Protect the Site Security endpoints and control who can give “critical” levels.

3. Design a real credential model with a life cycle, revocation and a link to an identity.

4. Build real access events, timestamped by the server and append-only.

5. Add integrity constraints, pagination, consistent HTTP errors, and strictly turn off demo data outside development.

17. Not verified

- no live HTTP execution is documented;
- no proof of a real deployment is available;
- an external reader or gate integration outside the repository cannot be ruled out;
- the exact business rules for “critical”, “Permanent” and visitors must be confirmed;
- the GDPR retention policy must be set by the responsible people;
- real performance and behavior under concurrency must be tested;
- the file:line references come from the initial audit and should be checked again if the code has changed since.

18. Conclusion

> The module must be considered a prototype that is not ready to be exposed in production until authentication, authorization, the credential model and event traceability are implemented and verified.